

Complements of Operational Research

5.3 Dynamic programming: workforce planning

José Firmino Aguilar Madeira

How many people to have each week?

Staffing needs vary from week to week (construction works, events, high season, campaigns). Two ways of losing money:

- having **too many people**: we pay people who are not needed;
- **hiring too often**: each hiring has a fixed cost (advert, selection, training, insurance) and a cost per person.

Keeping someone idle for a week may be cheaper than letting them go and hiring again.

The model (the one used in class)

- requirements b_t in week t ; never work with fewer;
- surplus: c_e per extra worker per week;
- hiring y people: $c_f + c_v y$ (only if $y > 0$);
- releasing workers costs nothing.

Question: how many workers to have each week to minimise the total cost?

The example: setting up fairs and conferences

An event set-up company needs, over the next 5 weeks:

Week t	1	2	3	4	5
Requirement b_t	3	8	9	4	6

Surplus: $c_e = 300$ € per worker per week.

Hiring: $c_f = 600$ € (fixed) + $c_v = 150$ € per worker hired.

At the start nobody is hired.

Two simple policies

- **Follow the requirements** (3, 8, 9, 4, 6): four hirings, no surplus $\Rightarrow$ 4050 €.
- **Hire 9 right at the start**: a single hiring, a lot of surplus $\Rightarrow$ 6450 €.

The optimum lies somewhere in between.

There are $7 \times 2 \times 1 \times 6 \times 4 = 336$ possible plans — DP finds the best one without looking at them all.

- **Stage:** week $t = 1, \dots, 5$.
- **State:** s = workers in the previous week (that is all that matters from the past).
- **Decision:** x_t = workers in week t , with $b_t \leq x_t \leq 9$ (it never pays to have more than the maximum requirement).
- **Cost of the week:**

$$c_t(s, x) = 300(x - b_t) + [x > s] (600 + 150(x - s))$$

($[x > s] = 1$ if we hire, 0 otherwise).

Recursion (backwards)

$$f_t(s) = \min_{b_t \leq x \leq 9} \{c_t(s, x) + f_{t+1}(x)\},$$

$$f_6(s) = 0; \text{ answer: } f_1(0).$$

Note that the state of a week is the decision of the previous one — just as in the shortest path the next node is the arc chosen.

The tables, from week 5 to week 1

Week 5 ($b_5 = 6$; state $s = x_4 \in \{4, \dots, 9\}$)

s	x_5^*	$f_5(s)$	
4	6	900	hire 2: $600 + 300$
5	6	750	hire 1: $600 + 150$
6–9	6	0	release the surplus

Week 4 ($b_4 = 4$; $s = x_3 = 9$)

x_4	4	5	6	7	8	9
$300(x - 4) + f_5(x)$	900	1050	600	900	1200	1500

$f_4(9) = 600$ with $x_4^* = 6$: keeping 2 extra avoids hiring in week 5.

Week 3 ($b_3 = 9$; $x_3 = 9$ necessarily)

$f_3(8) = 750 + 600 = 1350$; $f_3(9) = 0 + 600 = 600$.

Week 2 ($b_2 = 8$; $x_2 \in \{8, 9\}$)

s	$x_2 = 8$	$x_2 = 9$	$f_2(s)$
3	$1350 + 1350$	$1800 + 600$	2400

($x_2 = 8$: hire 5 = 1350; $x_2 = 9$: hire 6 = 1500, plus 300 of surplus.)

Week 1 ($s = 0$): $x_1 = 3$ costs $600 + 450 = 1050$;

$f_1(0) = 1050 + f_2(3) = \mathbf{3450}$.

(With $x_1 > 3$ we pay surplus and the total goes up.)

The optimal plan

Week	b_t	x_t^*	action	cost
1	3	3	hire 3	1050
2	8	9	hire 6	1800
3	9	9	—	0
4	4	6	release 3	600
5	6	6	—	0
			total	3450

Interpretation

- In week 2 we hire **one extra** (9 instead of 8): this avoids a third hiring in week 3.
- In week 4 we keep **two extra** (6 instead of 4): this avoids hiring again in week 5.
- Cost 3450 €, against 4050 € following the requirements (−15%) and 6450 € with 9 from the start.

What if the fixed hiring cost changes?

c_f (€)	optimal plan	cost (€)
0	3, 8, 9, 4, 6 (= requirements)	1650
300	4 tied plans	2850
600	3, 9, 9, 6, 6	3450
1200	3, 9, 9, 6, 6	4650
2000	9, 9, 9, 6, 6	6050

- With no fixed cost, we follow demand.
- The more expensive hiring is, the more people are “kept” between peaks.
- With $c_f = 2000$ €, the whole team is hired right in week 1.

With $c_f = 300$ there is a tie between 3–8–9–4–6, 3–8–9–6–6, 3–9–9–4–6 and 3–9–9–6–6: it is the boundary between the two logics.

```
def mao_obra(b, ce, cf, cv):
    n, M = len(b), max(b)
    def custo(s, x, t):
        contrata = cf + cv * (x - s) if x > s else 0
        return ce * (x - b[t]) + contrata
    f = {s: 0 for s in range(M + 1)}      #  $f_{n+1}$ 
    dec = [None] * n
    for t in range(n - 1, -1, -1):      # week  $n \rightarrow 1$ 
        estados = [0] if t == 0 else range(b[t - 1], M + 1)
        g, dec[t] = {}, {}
        for s in estados:
            x = min(range(b[t], M + 1),
                    key=lambda x: custo(s, x, t) + f[x])
            g[s], dec[t][s] = custo(s, x, t) + f[x], x
        f = g
    plano, s = [], 0
    for t in range(n):
        s = dec[t][s]; plano.append(s)
    return f[0], plano

print(mao_obra([3, 8, 9, 4, 6], 300, 600, 150))  # (3450, [3, 9, 9, 6, 6])
```

Check: enumerating the 336 plans gives the same optimum (and only that one).

The same scheme works for planning production and inventory (state = stock at the start of the period) or maintenance (state = age of the equipment) — it links to ch. 4 and ch. 7.

Example 7

A building contractor estimates that, over the next 5 weeks, they need 5, 7, 8, 4 and 6 workers, respectively. Surplus labour costs 300 m.u. per worker per week, and a new hiring, in any week, has a fixed cost of 400 m.u. plus 200 m.u. per worker. What is the optimal number of workers in each week?

Example 8

The same problem for 4 weeks, with requirements 5, 7, 4 and 6, surplus of 200 m.u. per worker per week and hiring with a fixed cost of 300 m.u. plus 200 m.u. per worker.

Workforce planning (homework)

A building contractor estimates that, over the next 6 weeks, they need 3, 5, 4, 6, 8 and 3 workers, respectively. Surplus labour costs 300 m.u. per worker per week, and a new hiring, in any week, has a fixed cost of 500 m.u. plus 200 m.u. per worker. What is the optimal number of workers in each of the weeks?

Submission: by email to the lecturer, by the deadline given in class. The other homework exercises (shortest path and knapsack) and the test and exam exercises: [Exercise sheet 5](#).

Practical assignment no. 2: create and solve 2 dynamic programming problems (report; groups of 2).

KEY TAKEAWAYS — 5.3

1. Stage = week; state = workers of the previous week; decision $x_t \geq b_t$.
2. $f_t(s) = \min_x \{c_e(x - b_t) + [x > s](c_f + c_v(x - s)) + f_{t+1}(x)\}$, backwards; answer $f_1(0)$.
3. The optimum balances surplus and hirings: here, hire one extra in week 2 and keep two extra in week 4.
4. The same three steps throughout the chapter: stages, state, recursion — shortest path, knapsack and workforce planning are the same method.

End of chapter 5. Next: 6 Queueing.

References and resources

References Full list: *Course bibliography*.

- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 11th ed., McGraw-Hill, 2021 — Dynamic Programming (allocation and planning examples).
- Taha, H. A., *Operations Research: An Introduction*, 10th ed., Pearson, 2017 — workforce size model.
- Kasana, H. S., Kumar, K. D., *Introductory Operations Research*, Springer, 2004 — ch. 12.

Resources

- Exercise sheet 5. `code/python/05_dynamic_programming/05_3_workforce/mao_de_obra.py`.